
Design Pattern Bericht

Observer Pattern (2-stufige Event-/Listener-Architektur)

Das Observer Pattern ist das zentrale Architekturmuster des Notification-Systems. Es implementiert eine zweistufige Event-/Listener-Architektur, die eine klare Trennung zwischen Domain-Logik und technischer Zustellung ermöglicht.

Implementierung

Stufe 1: Beobachtung von Domain-Events (fachliche Ereignisse)

Publisher:

- ProductService: Publiziert ProductEvent<?> bei Produktänderungen (Preis, Name, Beschreibung, Rabatt, Restock)
- OrderService: Publiziert OrderCompletionEvent nach erfolgreicher Zahlung

Observer (Listener Stufe 1):

- SubscriptionNotificationListener (src/main/java/at/qe/skeleton/listeners/SubscriptionNotificationListener.java)
- Beobachtet ProductEvent<?>
- Ermittelt betroffene Subscriptions
- Erstellt Notification-DB-Einträge
- Publiziert danach kanal-spezifische Delivery-Events (Stufe 2)
- OrderCompletionEventListener (src/main/java/at/qe/skeleton/listeners/OrderCompletionEventListener.java)
- Beobachtet OrderCompletionEvent
- Erstellt Notification-DB-Eintrag
- Publiziert danach Delivery-Event (Stufe 2)

Technische Details:

- @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT): Listener reagieren nur bei erfolgreichem DB-Commit
- @Async: Verarbeitung ist asynchron und entkoppelt vom Request

Stufe 2: Beobachtung von Delivery-Events (technische Zustellung)

Publisher:

- Listener aus Stufe 1 publizieren:
- EmailNotificationEvent
- SmsNotificationEvent
(abhängig vom NotificationType / Strategy)

Observer (Listener Stufe 2):

- EmailNotificationEventListener (src/main/java/at/qe/skeleton/listeners/EmailNotificationEventListener.java)
- Beobachtet EmailNotificationEvent<?>
- Delegiert an EmailNotificationService
- SmsNotificationEventListener (src/main/java/at/qe/skeleton/listeners/SmsNotificationEventListener.java)
- Beobachtet SmsNotificationEvent<?>
- Delegiert an SmsNotificationService

Code-Beispiele

```
// Publisher: OrderService publiziert OrderCompletionEvent
applicationEventPublisher.publishEvent(new OrderCompletionEvent(fullOrder));

// Stufe 1 Listener: OrderCompletionEventListener
@Async
@Transactional
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void handleOrderCompleteEvent(OrderCompletionEvent event) {
    Notification notification = notificationService.createNotification(
        order.getUser().getId(),
        NotificationType.EMAIL,
        event
    );
    applicationEventPublisher.publishEvent(new EmailNotificationEvent<>(notification,
event));
}

// Stufe 2 Listener: EmailNotificationEventListener
@Async
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void handleEmailNotificationEvent(EmailNotificationEvent<?> event) {
    emailNotificationService.sendEmail(event);
}
```

Factory Method Pattern

Das Factory Method Pattern wird im NotificationType Enum verwendet, um die Erstellung von Event-Objekten zu kapseln. Jeder Notification-Typ (EMAIL, SMS) hat eine eigene Factory-Methode, die das entsprechende Event-Objekt erstellt.

Implementierung

Creator (Factory):

- NotificationType Enum (src/main/java/at/qe/skeleton/model/NotificationType.java)
- Definiert die Factory-Methode createEvent()
- Jede Enum-Konstante (EMAIL, SMS) ist eine konkrete Factory

Product:

- EmailNotificationEvent und SmsNotificationEvent sind die konkreten Produkte
- Beide implementieren das NotificationEvent<?> Interface

Code-Beispiel

```
public enum NotificationType {
    EMAIL(EmailNotificationEvent::new),
    SMS(SmsNotificationEvent::new);

    private final BiFunction<Notification, Payload<?>, NotificationEvent<?>>
eventConstructor;

    NotificationType(BiFunction<...> eventConstructor) {
        this.eventConstructor = eventConstructor;
    }

    public NotificationEvent<?> createEvent(Notification notification,
        Payload<? extends PayloadInterface> payload) {
```

```

    return eventConstructor.apply(notification, payload);
}
}

```

Decorator Pattern

Das Decorator Pattern wird durch die Payload Klasse implementiert, die andere Objekte wrappt und deren Funktionalität erweitert, ohne die ursprüngliche Klasse zu modifizieren.

Implementierung

Component:

- PayloadInterface: Definiert die gemeinsame Schnittstelle

ConcreteComponent:

- Order, Product, Subscription etc. implementieren PayloadInterface

Decorator:

- Payload<T> wrappt ein Objekt vom Typ T und delegiert Methodenaufrufe

Code-Beispiel

```

public interface PayloadInterface {
    String getPayloadSubjectLine();
}

```

```

public class Order implements PayloadInterface {
    @Override
    public String getPayloadSubjectLine() {
        return "Order " + orderNumber + " completed";
    }
}

```

```

public class Payload<T extends PayloadInterface> implements PayloadInterface {
    T payloadInfo;

    public Payload(T payloadInfo) {
        this.payloadInfo = payloadInfo;
    }

    @Override
    public String getPayloadSubjectLine() {
        return payloadInfo.getPayloadSubjectLine();
    }
}

```

Iterator Pattern

Das Iterator Pattern wird implizit durch Java Streams und Collections verwendet, um über Sammlungen zu iterieren, ohne deren interne Struktur zu kennen.

Implementierung

Iterator:

- Java Stream API (java.util.stream.Stream)
- Collections implementieren Iterable<T>

Concrete Iterators:

- `.stream(), .forEach(), .map(), .filter(), .collect()`

Code-Beispiele

```
List<Subscription> subscriptions = subscriptionRepository
    .findByProductAndType(productId, type)
    .stream()
    .distinct()
    .toList();
```

```
cart.getItems().stream()
    .map(cartItem -> {
        return orderItem;
    })
    .collect(Collectors.toList());
```

```
orders.forEach(order -> {
    orderLifecycleService.applyResolvedStatus(order, now);
});
```

Facade Pattern

Das Facade Pattern wird durch die Service-Klassen implementiert, die komplexe Subsysteme (Repositories, andere Services, Event-Publisher) hinter einer vereinfachten Schnittstelle verbergen.

Implementierung

Facade:

- `OrderService`: Vereinfacht die komplexe Order-Verarbeitung
- `ProductService`: Vereinfacht Produkt-Operationen
- `CartService`: Vereinfacht Warenkorb-Operationen

Subsystem:

- `Repositories`, `Event-Publisher`, andere `Services`, `Mapper`

Code-Beispiel

`@Service`

```
public class OrderService {
    private final OrderRepository orderRepository;
    private final OrderLifecycleService orderLifecycleService;
    private final CartService cartService;
    private final ProductService productService;
    private final ApplicationEventPublisher applicationEventPublisher;
```

`@Transactional`

```
public Order createOrder(OrderCreateDTO orderCreateDTO) {
    // Komplexe Logik wird hinter einfacher Methode verborgen
}
```

```
}
```